Supplementary methods and complete results

A Coefficient-Aware Finite-Difference Benchmark for Solver Selection and CPU/GPU Stencil Scaling in Heat-Conduction Simulation

This companion retains the full experimental account underlying the compressed main paper: all nine result tables, six original figures, detailed protocols, failures and limitations. S-prefixed table, figure and equation numbers refer to this document. Experimental data and claims are unchanged; the main paper uses a smaller selection of displays. Full reference titles and identifiers are retained below.

S1. Mathematical Model and Discretisation

The model problem is the steady heat-conduction equation with homogeneous Dirichlet boundary conditions unless otherwise stated. The coefficient k(x, y) represents spatially varying thermal conductivity.

$$-\nabla\cdot(k(x,y)\nabla u(x,y)) = f(x,y),\ (x,y) \in (0,1)^2 \tag{S1}$$

The smooth test uses a sinusoidal variable coefficient, while the high-contrast stress test uses a circular inclusion. These two cases are deliberately synthetic: they remove geometry and data-acquisition uncertainty so that the numerical mechanisms can be measured cleanly.

$$k(x,y) = 1 + 0.5\sin(2\pi x)\sin(2\pi y) \tag{S2}$$

$$k(x,y) = 1 + 99\,\chi_D(x,y) \tag{S3}$$
$$D = \{(x,y) : (x-0.5)^2 + (y-0.5)^2 < 0.15^2\}$$

For the decision-map study, this pair is expanded into a parameterised coefficient grid. The smooth family uses k = 1 + a sin(2πx) sin(2πy) with a = (C_k - 1)/(C_k + 1), giving target contrast C_k. The inclusion, layered, and checkerboard families use piecewise values in {1, C_k} with C_k in {10, 30, 100}, while the smooth family uses C_k in {3, 10, 30} and the constant case gives Ck = 1. The layered field changes at y = 1/2; the checkerboard uses eight blocks per coordinate direction. The circle has radius 0.15 and centre (1/2, 1/2).

To report contrast, coefficient sharpness, and coefficient geometry separately, each coefficient field is summarised by C_k, the maximum nodal Euclidean magnitude of the log-gradient (centred interior and first-order one-sided boundary differences), and a grid-normalised total-variation proxy. In Equation (S4), $\Delta_x k$ and $\Delta_\gamma k$ denote unscaled nearest-neighbour differences over all nodal pairs in each coordinate direction, and their absolute sums are divided by N−1. For discontinuous coefficient fields, the log-gradient descriptor depends on grid spacing and is interpreted at fixed N rather than as a continuous invariant. Small-grid spectral evidence is also recorded through computed condition-number estimates $\kappa(A) = \lambda_{max}(A) / \lambda_{min}(A)$.

$$C_k = \frac{max(k)}{min(k)},\quad G_k^{(h)} = ||\nabla_h\, log(k)||_\infty \tag{S4}$$
$$V_k = \frac{\Sigma|\Delta_x k| + \Sigma|\Delta_y k|}{N-1}$$

The domain is discretised on an N by N uniform grid with spacing h = 1/(N - 1). Interior unknowns are ordered lexicographically. For an interior node (i, j), the conservative five-point stencil is assembled by face-based arithmetic averaging of adjacent conductivity values in the main two-dimensional decision map. Arithmetic averaging defines the primary two-dimensional benchmark operator; harmonic averaging probes sensitivity to the interface treatment. For discontinuous coefficient fields, harmonic face averaging is also implemented and tested as a sensitivity check because it matches the series-resistance interpretation of one-dimensional layered conduction. The shared positive face weights give a sum of weighted squared nodal differences in the discrete energy. With boundary values fixed, both choices produce sparse symmetric positive definite systems for positive k and homogeneous Dirichlet boundaries.

$$(Au)_{ij} = \tag{S5}$$
$$k_{i+1/2,j}\frac{u_{ij} - u_{i+1,j}}{h^2} + k_{i-1/2,j}\frac{u_{ij} - u_{i-1,j}}{h^2}$$
$$+ k_{i,j+1/2}\frac{u_{ij} - u_{i,j+1}}{h^2} + k_{i,j-1/2}\frac{u_{ij} - u_{i,j-1}}{h^2}$$

Arithmetic faces are retained to preserve the specified nodal graph-diffusion benchmark and its historical comparisons, not because they are the preferred interface law for composite heat conduction. Interface conductivity is a standard numerical heat-transfer issue [30]. For two half-links of equal length with conductivities a and b, continuity of flux gives the series resistance h/(2a) + h/(2b), hence kH = 2ab/(a + b), whereas kA = (a + b)/2. Their difference is $(a - b)^2/[2(a + b)]$: it is $O(h^2)$ for smooth positive conductivity, but need not be small at a jump. At contrast c = 100, $kA/kH = (c + 1)^2/(4c) = 25.5025$ on a mixed link. This is a local conductance ratio under the half-link interface assumption, not a global temperature-error estimate. Thus the arithmetic high-contrast results describe solver behaviour for that discrete operator; they do not validate composite-material heat flux. Harmonic averaging has the correct aligned series-resistance interpretation, while a curved interface requires additional geometric treatment. The matched-field audit includes both averages so that its conclusions are not restricted to arithmetic faces.

For smooth convergence verification, the exact solution is chosen in closed form and f is generated analytically for the constant and smooth coefficient cases. Errors are reported as the interior-node root-mean-square (normalised discrete L2) and maximum norms. The observed convergence rate is obtained by a least-squares fit of log(error) against log(h). A separate one-dimensional verification problem uses k(x) = k1 for x < 1/2 and k(x) = k2 for x > 1/2 with u(0) = 0 and u(1) = 1. Its analytic solution is piecewise linear and satisfies continuity of both u and k ux at the interface; this test isolates whether arithmetic and harmonic face averages respect the expected interface flux.

$$u_{exact}(x, y) = \sin(\pi x)\sin(\pi y) \tag{S6}$$

S2. Solver and Implementation Methods

The assembled sparse matrix is solved with three methods: unpreconditioned CG, Jacobi-preconditioned CG, and algebraic-multigrid preconditioned CG. The AMG case uses the tested PyAMG default smoothed-aggregation hierarchy as a V-cycle preconditioner for SciPy CG; this should be read as one concrete AMG configuration rather than a claim about all AMG parameter choices. The recorded PyAMG 5.3.0 defaults use symmetric strength, standard aggregation, Jacobi prolongation smoothing with omega = 4/3, symmetric block Gauss-Seidel pre/post smoothing, at most ten levels, and a coarse-grid limit of ten unknowns. Historical runs did not fix the random starts used in PyAMG spectral-radius estimation, so exact hierarchy-dependent results can vary across reruns. Manufactured-solution runs use a relative residual tolerance of 1e-11 so that solver error does not dominate discretisation error; solver-comparison runs use SciPy CG with relative tolerance 1e-8, zero absolute tolerance, the default zero initial guess, and a maximum of 12000 iterations in the decision map. The baseline coefficient-family solver comparisons use the same deterministic unit right-hand side at a given grid size, so the right-hand side is controlled across cases. The controlled extension below separately varies the right-hand side, which can excite different spectral components. The reported time separates preconditioner setup from iterative solve time and also gives the total time. Matrix assembly and right-hand-side construction are excluded from these times. Iterative solve times include a callback that recomputes the true relative residual after every iteration, adding one sparse matrix-vector product and norm evaluations per iteration. The reported selections and speedups therefore apply to this monitored implementation. Methods are run in a fixed order, and the first AMG setup in a fresh process also includes the PyAMG import; these timings do not isolate hierarchy construction from first-use overhead. BLAS thread settings and processor power states were not recorded, limiting exact performance replication. Failed convergence would be recorded as a structured row in the raw CSV artefact, although all baseline benchmark cases converged.

Three evidence groups have distinct timing meanings [31]. Historical coefficient-family results include a true-residual evaluation at every iteration, fixed method order and the first PyAMG import inside setup; their thread settings were not recorded. The controlled matched-field extension uses warm imports, shuffled order, one BLAS thread, paired setup seeds and count-only callbacks, with rtol = 1e-9 and independently checked residual at most 1e-8. The new callback calibration holds the arithmetic operator, unit source, three candidate methods, rtol = 1e-8, warm imports, one BLAS thread and setup seeds fixed, changing only the callback within each pair. All three groups exclude matrix assembly and source construction, and include fresh preconditioner setup in total time. Historical speedups are monitored-implementation results; translation reversals and empirical cost floors are within-extension comparisons. Neither time ratios nor winner boundaries are pooled across these groups. The calibration estimates callback sensitivity under its stated conditions; it cannot retrospectively separate historical import, ordering, threading or machine-state effects.

Matrix-free performance is measured with a separate stencil apply matching Equation (S5). Three CPU paths are compared: NumPy vectorisation, Numba serial compilation, and Numba parallel compilation with controlled thread counts. Throughput is reported as an estimated stencil bandwidth using a fixed byte model: 11 double-precision values per interior point for the variable-coefficient stencil, or 88 bytes per point. The Jacobi crossover uses five double-precision values per interior point, or 40 bytes per point. These estimates are operational metrics for comparing implementations of the same stencil family, not hardware peak-bandwidth or direct DRAM-transaction claims.

The CUDA experiment uses a repeated Jacobi update rather than the full variable-coefficient operator. This isolates a simple memory-bound stencil and avoids conflating kernel throughput with sparse-solver algorithmics. CPU and GPU arrays are allocated once, one batch is used for warm-up, and timing continues for at least three batches and 0.2 s. CUDA uses 16 by 16 thread blocks and the Numba CPU baseline uses four threads. These are within-run averages, not independent timing replicates. CUDA timings use a host wall-clock timer around resident-data batches of 20 updates, including Python launch overhead and device synchronisation after each batch. Allocation, compilation warm-up, and host-device transfers are excluded. This boundary matches repeated time-stepping or smoother-like updates where data remain resident on the device, but it should not be read as end-to-end application speedup.

For a single solve, the decision map compares setup-plus-solve times retrospectively: all candidates have already been run, and the winning time excludes evaluation of the other candidates. It selects the fastest converged solver by setup-plus-solve time, making one-time AMG setup cost visible rather than treating iteration count as the only outcome. It is not a repeated-right-hand-side reuse model; such a model would require an explicit solve count m. We report Spearman rank correlations between Ck, Gk(h), Vk, the available-grid condition estimate, CG iterations, and the speedup of the selected solver over CG. The coefficient set is a designed benchmark grid rather than a random sample from a defined population, so rank correlations are treated as descriptive association diagnostics rather than population-level inferential estimates. Pooled rows summarise all case-size entries but are not treated as independent samples because the same coefficient family appears at multiple grid sizes. Fixed-N rows provide the safer coefficient-level reading and include deterministic case-resampling intervals. Iteration-count associations are treated as the primary solver-difficulty diagnostic; associations with selected-solver speedup are secondary and exploratory because the full decision grid uses a single timing pass per cell. Permutation p-values are archived in the CSV as diagnostics rather than used to claim pairwise descriptor separation.

$$s^*(c, N) = arg\ min_{s \in S} [T_{setup}(s, c, N) + T_{solve}(s, c, N)] \quad \text{(S7)}$$

Timing stability is assessed by repeating selected decision-map cases five times and then reporting the solver chosen by median total time, the per-repeat vote fraction, and the relative interquartile range of the selected method. The full decision grid remains a single-pass benchmark, while the repeat check probes whether the most important decisions are robust or near ties.

The hardware crossover model is similarly bounded: for the repeated Jacobi kernel it fits $T(n) = \alpha + \beta n$, where $n = (N - 2)^2$, and archives the global four-point fit as an exploratory summary rather than a validated local crossover model.

S3. Experiments and Results

All experiments were run on Windows 11 with Python 3.12.9, NumPy 2.4.6, SciPy 1.18.1, Numba 0.67.0, PyAMG 5.3.0, and Numba-CUDA 0.30.4. The machine used an AMD Ryzen 9 7940HS w/ Radeon 780M Graphics CPU with 8 physical cores and 16 logical processors, approximately 16 GB of system memory reported by the operating system as 15.2 GB usable, and an NVIDIA GeForce RTX 4060 Laptop GPU with compute capability 8.9. The exact raw CSV outputs, environment metadata, and scripts are included in the supplementary artefact.

The decision map covers 13 coefficient cases, three grid sizes, and three Krylov/preconditioner choices. The spectral conditioning study uses N = 32, 64, and 128. Extremal eigenvalues of the unpreconditioned matrix are estimated with SciPy eigsh (smallest/largest algebraic eigenvalue, tolerance 1e-6, maximum 20000 iterations). These estimates do not describe the preconditioned operator; the N = 256 correlations using an N = 128 spectral proxy are cross-resolution diagnostics. The rank-correlation analysis uses all 39 case-size decisions and three fixed-grid strata of 13 cases; condition numbers are matched to N = 64 and N = 128 where available, and the N = 128 condition estimate is used as the maximum-available case-level spectral proxy for N = 256. The repeated timing check uses five repeats for one constant case and four nonconstant representative cases at each of the three decision-grid sizes, covering 15 case-size cells. The face-averaging sensitivity study reruns the three discontinuous

Ck = 100 stress cases with arithmetic and harmonic face averages at N = 64, 128, and 256. The hardware crossover fit used the measured Jacobi-kernel timings at N = 512, 1024, 2048, and 4096.

S3.1. Verification and Coefficient Difficulty

The manufactured-solution study confirms the expected second-order behaviour of the finite-difference scheme. Table S1 reports fitted rates using N = 32, 64, 128, and 256 with a CG residual tolerance of 1e-11. Both the constant and smooth variable-coefficient cases produced L2 rates close to two. Figure S1 gives the same result as a log-log error plot.

Table S1. Manufactured-solution convergence for the finite-difference discretisation.

Case	p_L2	p_inf	Finest L2	Residual
Constant coefficient	2.01	2.00	6.35e-6	1.60e-12
Smooth variable coefficient	2.02	2.00	6.39e-6	2.01e-11

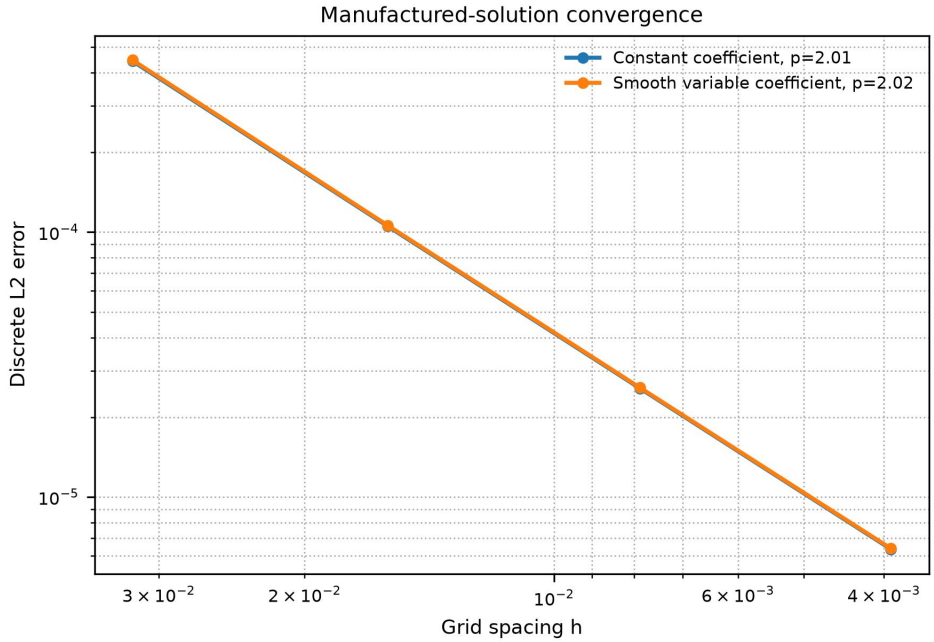


Figure S1. Manufactured-solution convergence of the finite-difference operator.

The discontinuous-interface check in Table S2 isolates a different error mechanism. For an aligned one-dimensional two-material conduction problem, harmonic face averaging reproduced the analytic interface flux to roundoff, while arithmetic averaging showed a grid-refined interface error. This does not make harmonic averaging a universal choice for all multidimensional discontinuities, but it prevents the benchmark from relying only on smooth manufactured-solution evidence.

Table S2. Discontinuous-interface verification on a one-dimensional two-material heat-conduction problem. Harmonic face averaging recovers the analytic flux continuity to roundoff in this aligned-interface test.

N	L2 arithmetic	L2 harmonic	Flux err. arithmetic	Flux err. harmonic
32	1.27e-2	5.96e-16	6.33e-2	5.11e-15
64	6.21e-3	1.56e-15	3.07e-2	6.66e-15
128	3.07e-3	3.55e-14	1.51e-2	2.44e-14
256	1.53e-3	9.51e-15	7.49e-3	4.91e-14

Table S3 reports representative coefficient descriptors. The spectral evidence confirms that contrast alone is not a complete difficulty measure. At N = 128, the inclusion case with Ck = 100 had a computed condition-number estimate of 6.43e5, while the checkerboard case with the same contrast had a much smaller estimate of 3.12e4 under this discretisation. Figure S2 shows the same effect across the measured coefficient grid.

Table S3. Representative coefficient-difficulty descriptors. Metrics use N = 64 for the discrete descriptors; kappa(A) is the largest available spectral estimate, here N = 128.

Case	Family	Ck	Gk(h)	TV proxy	kappa(A)
constant	Constant	1.0	0.00	0.00	6.54e+3
smooth_c30	Smooth	29.7	16.41	4.76	1.38e+4
inclusion_c100	Inclusion	100.0	205.15	113.14	6.43e+5
layered_c100	Layered	100.0	145.06	100.57	2.61e+5
checkerboard_c100	Checkerboard	100.0	205.15	1408.00	3.12e+4

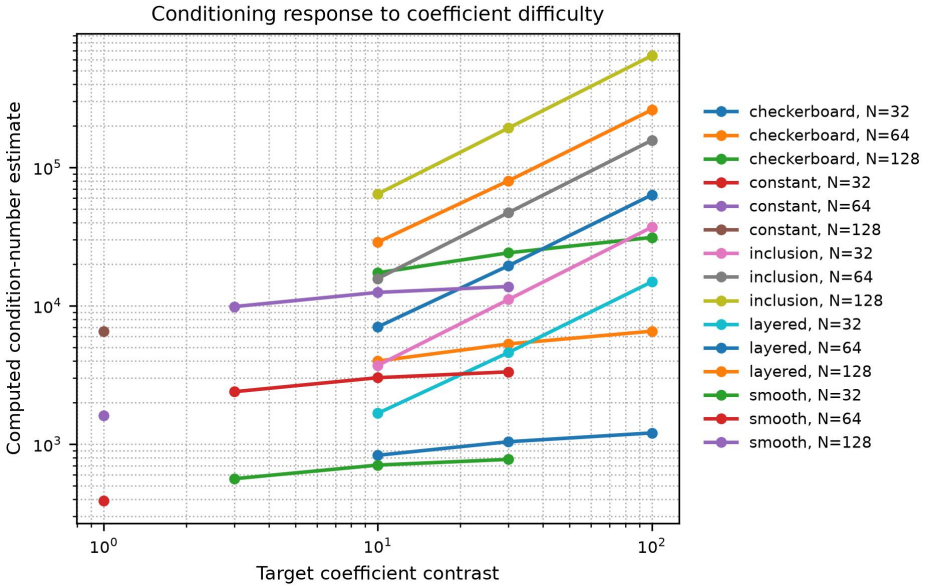


Figure S2. Computed condition-number response to coefficient contrast and coefficient geometry.

The supplementary rank-correlation table distinguishes pooled size effects from fixed-grid coefficient comparisons. The matched condition estimate has pooled correlation 0.94 with selected-solver speedup, but fixed-grid samples contain only 13 cases and their case-resampling intervals overlap. These are descriptive ordering signals, not population confidence guarantees. The matched-field audit below tests descriptor sufficiency directly, rather than inferring it from pooled correlation.

S3.2. Solver Decision Map and Preconditioner Effect

The single-pass setup-inclusive decision map records a resolution-dependent outcome. Within each grid size the winning class is the same across all tested coefficient fields, so this map supplies no evidence that descriptors improve prediction of the winner beyond resolution alone. Over the 13 coefficient cases, Jacobi-PCG was fastest in every N = 64 run, while AMG-PCG was fastest in every N = 128 and N = 256 run. Table S4 summarises this transition. The median speedup of the selected solver over CG increased from 2.79x at N = 64 to 7.80x at N = 256, and the largest speedup reached 25.72x for the layered Ck = 100 case. Because each decision-map cell uses one timing pass, this map is interpreted together with the repeated representative cases below.

Table S4. Setup-inclusive single-solve decision summary over the coefficient-family grid. Counts report the fastest converged method by setup-plus-solve time from one timing pass per cell.

N	Cases	CG	Jacobi-PCG	AMG-PCG	Median speedup	Max speedup
64	13	0	13	0	2.79	6.76
128	13	0	0	13	5.44	19.40
256	13	0	0	13	7.80	25.72

The complete arithmetic/harmonic sensitivity results are reported below in Table S6.

The supplementary timing-stability table reports all 15 case-size cells in the existing repeated-timing dataset. The median-best solver also won all five individual repeats in 14 cells: Jacobi-PCG for the four nonconstant N = 64 cases, and AMG-PCG for all five cases at both N = 128 and N = 256. The constant N = 64 cell is a near tie: the median favours Jacobi-PCG by only 1.03x, while CG wins four repeats. For constant conductivity on this uniform grid, the diagonal is a scalar multiple of the identity; Jacobi therefore does not improve the spectral condition

number and yields equivalent CG iterates in exact arithmetic. The archived single-pass CG and Jacobi-PCG iteration counts agree at all three sizes. Their small-grid timing difference is not evidence of an iteration-reduction benefit.

Solver behaviour changed substantially when the coefficient field became high contrast. At N = 256, unpreconditioned CG required 4370 iterations in the high-contrast case, compared with 770 iterations in the smooth case. Jacobi preconditioning reduced the high-contrast iteration count to 535, while AMG-PCG reduced it to 12. Total time fell from 6.155 s for CG to 0.288 s for AMG-PCG in the high-contrast case. Table S5 summarises the finest-grid comparison, and Figure S3 shows total-time scaling across grid sizes. The iteration-scaling figure is retained in the supplementary artefact.

Table S5. Solver comparison at N = 256 grid points in each direction.

Case	Method	Iter.	Time (s)	Rel. residual
Smooth variable	CG	770	0.939	9.71e-9
Smooth variable	Jacobi-PCG	605	0.829	9.88e-9
Smooth variable	AMG-PCG	9	0.240	4.82e-9
High contrast	CG	4370	6.155	9.95e-9
High contrast	Jacobi-PCG	535	0.728	9.14e-9
High contrast	AMG-PCG	12	0.288	4.44e-9

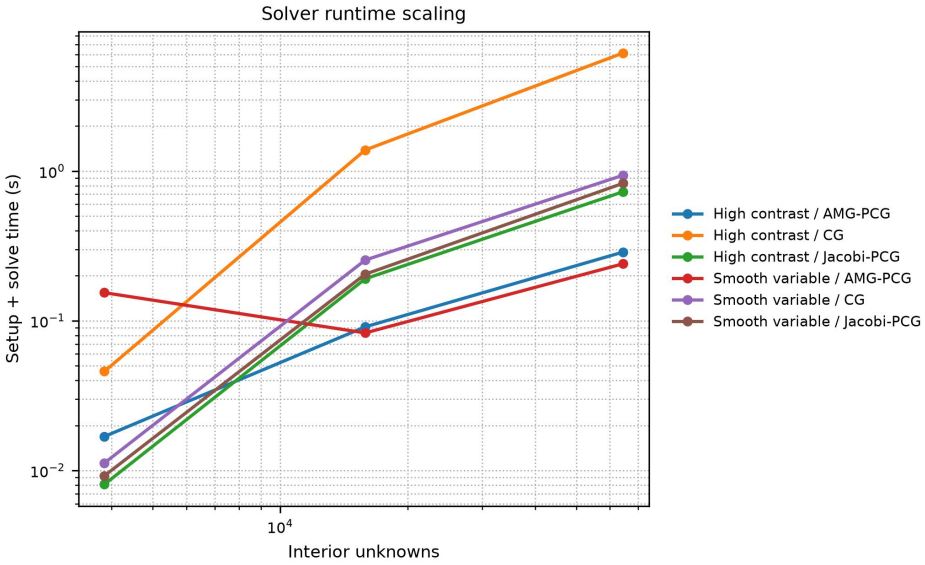


Figure S3. Total setup plus solve time for CG, Jacobi-PCG, and AMG-PCG.

The complete face-averaging check in Table S6 reports all nine historical case-size cells. Arithmetic/harmonic iteration counts differ substantially, including AMG changes for checkerboard fields, although the setup-inclusive winner is unchanged in these single passes. Agreement of winners does not establish physical equivalence of the operators, timing stability, or identical solution accuracy. The aligned one-dimensional verification and the two-dimensional solver-cost sensitivity answer separate questions.

Table S6. Complete contrast-100 face-averaging check under the historical monitored protocol. Entries are arithmetic/harmonic iteration counts and setup-inclusive winners (J = Jacobi-PCG, SA = AMG-PCG). These are single timing passes, not uncertainty estimates.

Case	N	CG	J	SA	Winner
Inclusion	64	796/725	129/129	10/10	J/J
Inclusion	128	1963/1891	264/263	12/11	SA/SA
Inclusion	256	4370/4326	535/535	12/12	SA/SA
Layered	64	1136/1120	164/181	9/9	J/J
Layered	128	2447/2442	325/367	10/10	SA/SA
Layered	256	5055/5054	657/715	10/10	SA/SA
Checkerboard	64	443/557	171/190	13/28	J/J
Checkerboard	128	1050/1308	350/388	17/32	SA/SA
Checkerboard	256	2268/2809	718/783	17/31	SA/SA

S3.3. Paired Callback Calibration

A prospective calibration covers constant, inclusion and checkerboard conductivity (contrast 100 for the latter two), N = 64, 128, 256, three methods, two callback modes and seven repeats: 378 recorded solves. Each repeat shuffles all 54 jobs; each solve constructs a fresh preconditioner with a seed matched across callback modes. Every run satisfies the independent residual threshold of 1e-8. All 189 paired solutions have identical byte hashes and iteration counts. Median paired monitored/count-only solve-time ratios range from 1.54 to 1.84 for CG, 1.58 to 1.77 for Jacobi-PCG, and 1.03 to 1.06 for AMG-PCG. These method-dependent penalties change reported speedups even when the median winner is unchanged.

Table S7 reports every case-size cell, including vote instability. The median winner agrees between callbacks in all nine cells. At constant N = 256, however, count-only AMG wins only three of seven repeats despite having the lowest method median; the median winner and majority vote answer different questions. The archived method summaries retain setup, solve and total-time quartiles for all candidates. Several N = 128 winners differ from the historical map under both callback modes, so callback removal alone cannot explain the historical-to-calibration difference. No universal N = 128 AMG crossover is inferred. The calibration does not measure cold-import overhead or control operating-system, power or thermal variation.

Table S7. Paired callback calibration. Winner entries give method, median total milliseconds and votes out of seven. Last columns give median paired monitored/count-only solve-time ratios. J = Jacobi-PCG; SA = AMG-PCG.

Case	**N**	**Monitored**	**Count-only**	**CG**	**J**	**SA**
Constant	64	CG 5.64 (7/7)	CG 3.55 (5/7)	1.58	1.58	1.05
Constant	128	CG 31.22 (6/7)	CG 19.11 (7/7)	1.67	1.70	1.04
Constant	256	SA 141.18 (7/7)	SA 147.70 (3/7)	1.84	1.76	1.04
Inclusion	64	J 6.67 (7/7)	J 4.12 (7/7)	1.63	1.62	1.03
Inclusion	128	J 38.28 (7/7)	J 22.34 (7/7)	1.72	1.61	1.04
Inclusion	256	SA 156.23 (7/7)	SA 152.34 (6/7)	1.79	1.77	1.03
Checkerboard	64	J 8.69 (6/7)	J 5.63 (7/7)	1.54	1.59	1.04
Checkerboard	128	J 50.18 (5/7)	J 29.65 (7/7)	1.73	1.65	1.06
Checkerboard	256	SA 192.31 (7/7)	SA 180.24 (7/7)	1.79	1.67	1.06

S3.4. Matched-Field and Forcing Audit

Matched fields. Two 2-by-8 binary motifs contain 12 high cells and 20 high/low boundary edges but have one and two connected components, respectively. Integer dilation by $s = N/16$ and low-conductivity padding embed each motif in the unit square. The high phase has $k = c$ and the background $k = 1$. For the tested grids, both fields have $12s^2$ high nodes, mean $1 + (c - 1)12s^2/N^2$, total variation $(c - 1)20s/(N - 1)$, and maximum log-gradient $\sqrt{2}\,(N - 1)\log(c)/2$. These counting identities are checked after embedding. A one-node translation preserves these quantities, component sizes, shape, and topology; it changes position relative to the source and boundary. Matching these invariants does not isolate topology as the sole cause of a performance difference.

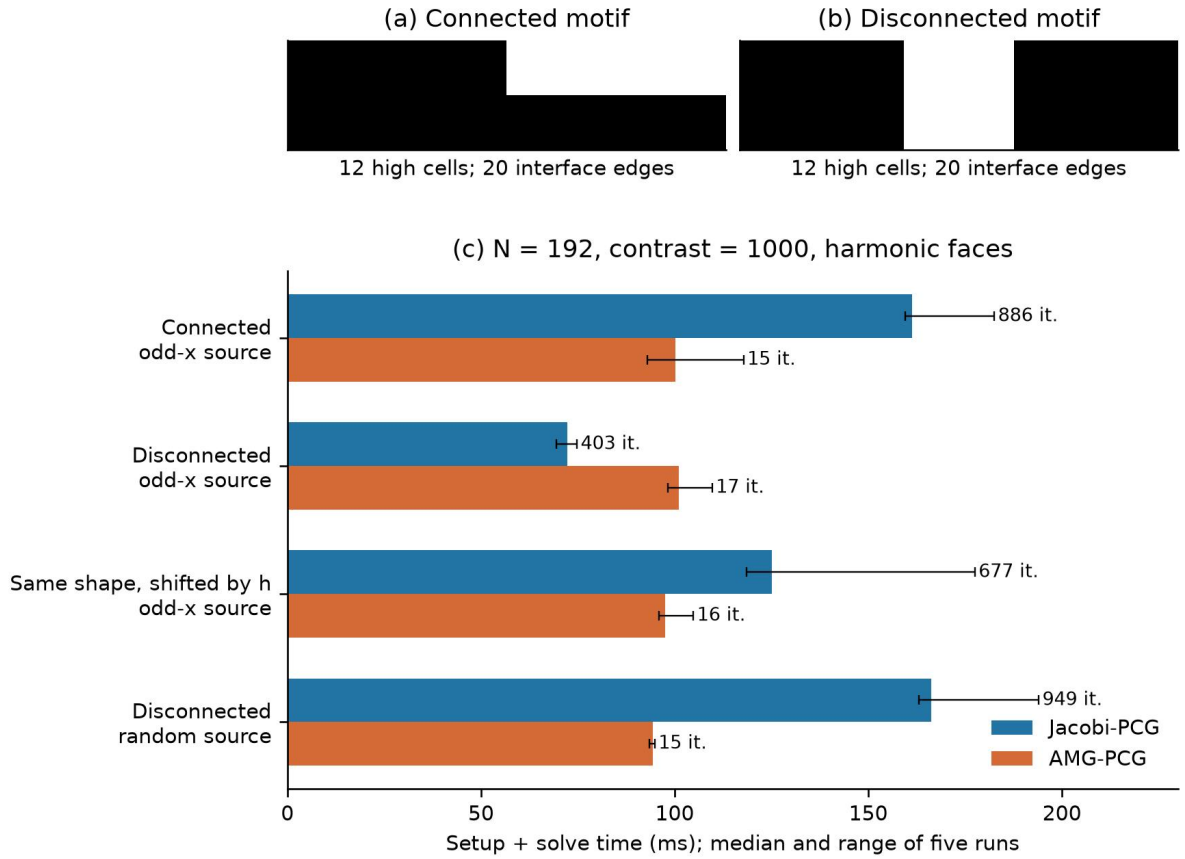


Figure S4. Matched coefficient motifs and the forcing/translation audit. Bars show median setup-plus-solve times; whiskers span five runs. Iteration counts are annotated. The fields share scalar coefficient descriptors, while a rigid translation also preserves shape and connectivity.

Controlled protocol. Following an exploratory 576-solve screen, confirmation uses c = 1000 and N = 96, 128, 192; centered, jointly transposed, and one-node x-shifted configurations; arithmetic and harmonic faces; and four unit-norm sources: constant, sin(2 pi x)sin(pi y), sin(pi x)sin(2 pi y), and a fixed-seed Gaussian vector. Jacobi-PCG and default SA-PCG each use five fresh setups per case, paired recorded setup seeds, shuffled method/source order, warm imports, and one BLAS thread. SA uses symmetric strength with theta = 0 and retains hierarchy diagnostics (keep=True). Count-only callbacks avoid the baseline residual-monitor overhead. CG uses rtol = 1e-9, zero initial guess, and at most 8000 iterations; an independent true relative residual must be at most 1e-8. Assembly and source construction are shared and excluded. The two timing protocols are not pooled.

A separate 3-by-6 motif family samples four connected and four disconnected patterns before observing their solver outcomes, retaining 12 occupied cells and 20 interface edges. It supplies 640 solves at N = 128, 192. This is a balanced constructed family, not a random sample of engineering materials. An additional 132-solve perturbation study varies translation direction and source admixtures with three repeats. The 1440 confirmation, 640 independent-family, and 132 perturbation solves all pass the true-residual check; the largest residual is below 3e-9. The pilot archive retains 84 failed acceptance checks, mainly at higher contrast or the CG iteration limit. No failed pilot timing is relabelled as a successful solve.

Mechanism. Let R reflect interior nodes in x and let D be the diagonal of A. If RA = AR, then R also commutes with D and the symmetrically scaled matrix S. For an odd source Rb = -b, every vector in the zero-start Krylov space generated by S and the scaled source g is odd. It is orthogonal to x-even eigenvectors in exact arithmetic. The centered two-island field satisfies this identity; its two slowest Jacobi-scaled modes are x-even. At N = 64 and c = 10000, computed squared source overlaps are below 1e-30 for both face averages. A transpose of both operator and source preserves Jacobi iteration counts, whereas translating the field in x breaks the reflection identity. This is a standard invariant-subspace explanation, not new convergence theory.

$$S = D^{-1/2} A D^{-1/2}, \quad g = D^{-1/2} b, \quad Rb = -b \tag{S8}$$

Figure S4 shows a concrete decision consequence at N = 192 with harmonic faces and c = 1000. For the odd-x source, the centered two-island field takes 403 Jacobi iterations and 72.2 ms, versus 17 AMG-PCG iterations and 101.0 ms. A one-node x translation increases Jacobi to 677 iterations and 125.1 ms, while AMG-PCG takes 16 iterations and 97.5 ms. Each winner is unchanged across the five repeats. With the same centered operator but a random source, Jacobi takes 949 iterations and 166.2 ms, and AMG-PCG takes 15 iterations and 94.3 ms. These are measured medians on this workstation, not universal crossover thresholds.

The full confirmation has a consistent per-repeat winner in 124 of 144 cells; the independent motif family has 55 of 64. Only two of the 48 centered-versus-x-shifted pairs reverse their winner consistently in all five repeats, so the reversal is a counterexample rather than a population-frequency claim. For the odd-x example within the Jacobi/SA portfolio, any deterministic rule restricted to the preserved coefficient invariants must choose the same method for both positions. Let T(j,s) be the median setup-plus-solve time for position j and candidate s in portfolio P. The least excess cost of a fixed choice across the equally weighted pair is E(P), defined below. For the two-method example, 1 + E(P) = 1.142. This empirical floor is not a statistical lower confidence bound, a bound for other solver portfolios, or a proposed selector. The raw tables include all pairs, unstable cells, and both methods.

$$E(P) \;=\; \min_{s \in P} \left[\frac{1}{2} \sum_{j=0,1} \frac{T(j, s)}{\min_{t \in P} T(j, t)} \right] \;-\; 1 \tag{S9}$$

Source perturbations test the interpretation without changing A: at N = 192, adding a constant component of amplitude 1e-8 to the unit odd source raises Jacobi from 403 to 569 iterations under the same stopping tolerance. A y translation preserves x reflection but can change the remaining spectrum and other symmetry sectors; parity alone is therefore not a complete runtime predictor. Offline spectra use symmetric Jacobi scaling and the nonsymmetric operator BA for the AMG inverse preconditioner B. Small-grid dense symmetrisation checks validate the numerical spectral procedure. Neither eigenvalue computations nor the reflection audit is included in a claimed low-cost selection policy.

Candidate-solver sensitivity. A separate 240-solve screen adds SA with symmetric-strength theta = 0.25 and classical Ruge-Stueben AMG with classical strength theta = 0.25. A subsequent 504-solve confirmation compares Jacobi, SA with theta = 0, and classical AMG at N = 128, 160, 192, using the two-island motif and the previously tested symmetric 3-by-6 anchor, whose three rows are each 110011. Each of seven repeats interleaves both field positions, both sources, both motifs, and all three methods at a fixed N. All 744 solves pass the same residual acceptance check; their timings are not pooled with earlier stages. The confirmation has 17 of 24 cells with the same winner in every repeat, and three of 12 translated pairs reverse their winner in all seven repeats. Table S8 reports all 12 pairs, including unstable cells and zero floors. For the original motif at N = 128 with odd-x forcing, centered Jacobi and classical AMG take 24.3 and 36.8 ms; after translation they take 42.3 and 34.2 ms. Jacobi iterations rise from 266 to 450, while classical AMG takes 11 in both positions. The three-method empirical excess-cost floor for this pair is 11.91%. At N = 192, classical AMG has the lowest median time at both positions, giving zero median-based floor for that pair, although the centered winner is not identical in all repeats. Thus translation sensitivity survives a stronger baseline, but its measured decision boundary and cost consequence depend on the candidate solver set.

Table S8. All 12 translated pairs in the interleaved confirmation. Winners use median total time: J = Jacobi-PCG, RS = classical AMG-PCG; SA was also a candidate. Stable means both positions retain their respective winner in all seven repeats. E(P) uses measured medians even for unstable pairs; it is not a confidence bound.

N	Motif	Source	Centered	Shifted	Stable	E(P) (%)
128	3x6	Odd-x	J	RS	Yes	12.42
128	3x6	Random	RS	RS	Yes	0.00
128	2x8	Odd-x	J	RS	Yes	11.91
128	2x8	Random	RS	RS	Yes	0.00
160	3x6	Odd-x	J	RS	Yes	9.69
160	3x6	Random	RS	RS	No	0.00
160	2x8	Odd-x	J	RS	No	4.86
160	2x8	Random	RS	RS	Yes	0.00
192	3x6	Odd-x	RS	RS	No	0.00
192	3x6	Random	RS	RS	Yes	0.00
192	2x8	Odd-x	RS	RS	No	0.00
192	2x8	Random	RS	RS	Yes	0.00

S3.5. CPU and CUDA Stencil Scaling

At N = 2048, NumPy required 0.1813 s per apply, while Numba serial required 0.005325 s and Numba parallel with four threads required 0.003725 s. Increasing thread count beyond four did not improve the largest case in this run; the measurements alone do not distinguish memory-system limits from scheduling overhead.

The CUDA Jacobi benchmark measures the resident-data update cost. At N = 512, GPU and CPU kernel times were essentially equal. At N = 1024, CUDA reached 1.52x speedup. At N = 2048 and N = 4096, the speedups increased to 3.23x and 3.51x, respectively. The measurements describe this kernel and workstation. They do not imply the same speedup for an end-to-end sparse solver.

Table S9. Representative stencil-throughput measurements. CPU entries use the variable-coefficient operator at N = 2048; CUDA entries are kernel-only Jacobi steps with arrays resident on the device.

Experiment	**Method**	**N**	**Time**	**Throughput/speedup**
CPU stencil	NumPy	2048	0.1813 s/apply	2.03 GB/s
CPU stencil	Numba serial	2048	0.005325 s/apply	69.18 GB/s
CPU stencil	Numba parallel, 4 threads	2048	0.003725 s/apply	98.90 GB/s
GPU crossover	CUDA kernel	2048	7.97e-4 s/step	3.23x vs CPU
GPU crossover	CUDA kernel	4096	3.15e-3 s/step	3.51x vs CPU

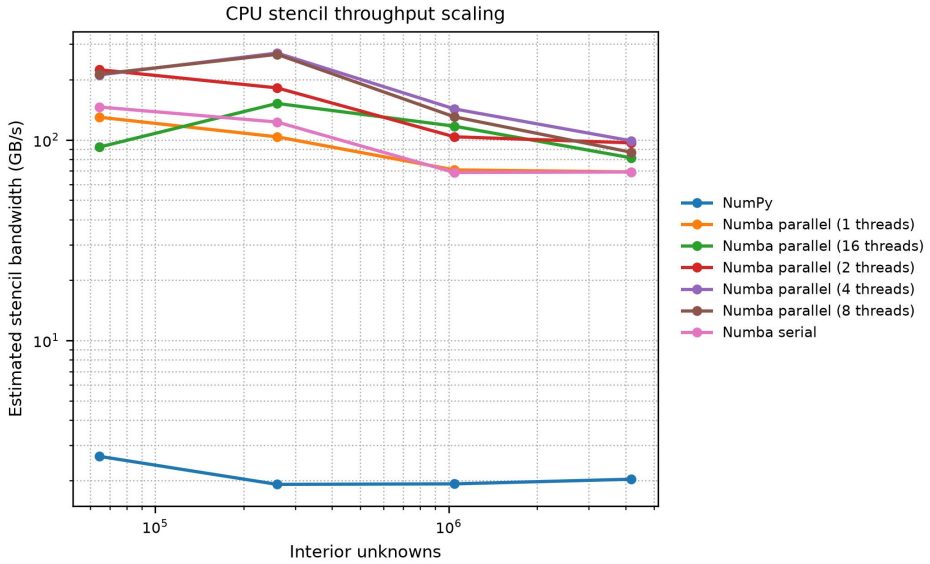


Figure S5. Estimated CPU stencil bandwidth for the variable-coefficient stencil apply.

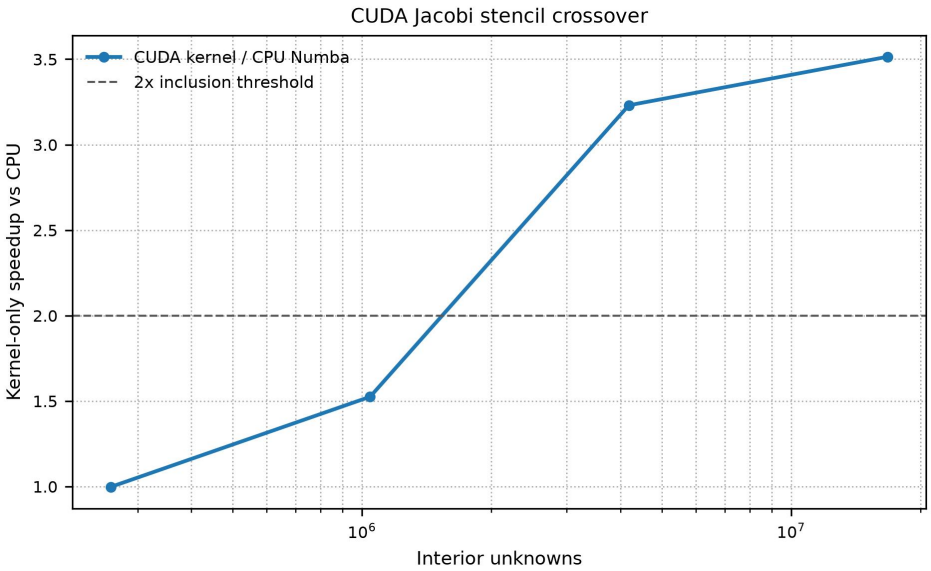


Figure S6. CUDA kernel-only speedup for repeated Jacobi updates relative to a four-thread Numba CPU baseline.

The measured timings are approximately equal at N = 512, and CUDA is faster at N = 1024. The supplementary artefact retains the exploratory four-point linear fit. Its negative CPU-time prediction at the measured N = 512 point invalidates its use as a crossover-location estimate; no fitted hardware-selection threshold is claimed.

S4. Discussion

The matched-field audit qualifies the baseline resolution-dependent decision map. The original monitored runs show the expected setup-versus-iteration trade-off, but their uniform winner at each resolution does not establish descriptor sufficiency. In the controlled extension, a rigid translation or a source perturbation changes the effective Krylov workload while retaining the listed coefficient invariants. The practical recommendation is to test forcing, alignment, and candidate-solver sensitivity before interpreting a coefficient-stratified timing map as a reusable decision rule. The classical-AMG check shows why a cost floor established for two methods must be recomputed when the portfolio changes. Symmetric sources can be physically appropriate; their fast solves should be reported as workload-specific evidence, not treated as an error or extrapolated to arbitrary forcing.

For physical composite-conduction use, the face law must be chosen from the interface geometry and resistance model before comparing solvers. The arithmetic benchmark is not evidence of accurate cross-interface flux at high contrast. For performance use, the callback calibration shows that an iteration monitor can disproportionately penalise inexpensive iterations; solver rankings and their uncertainty must therefore be reported with the callback, setup and execution protocol.

The coefficient descriptors are empirical summaries, not conditioning theory. Their uncertainty intervals overlap, and small fixed-grid strata do not establish a reliable ordering of predictive importance. Pooled correlations also reflect resolution; N = 256 uses an N = 128 condition estimate. Case-resampling intervals describe sensitivity to this constructed dataset, not population confidence or performance on unseen geometries.

The stencil measurements demonstrate implementation and size effects. Nonmonotone CPU scaling is consistent with bandwidth or scheduling limits, but no hardware counters separate these causes. CUDA timings include launch and final synchronisation with data resident on the device. They exclude transfer costs and full sparse solves; the four-point fit cannot validate a crossover threshold.

The scope is two-dimensional structured grids, synthetic coefficients and one workstation. Harmonic averaging does not resolve curved interfaces cutting grid cells. Industrial geometries, unstructured finite elements and transient multiphysics remain outside the evidence. The contribution is a reproducible audit of numerical correctness and conditional solver costs, not production solver robustness.

S5. Reproducibility Artefact

The previously published baseline and controlled extension are archived at https://doi.org/10.5281/zenodo.22668106, with the earlier baseline at https://doi.org/10.5281/zenodo.22303525. The present revision additionally supplies the callback calibration in results/raw/protocol_sensitivity, its prospective protocol in experiments/protocol_sensitivity.md, and complete generated summaries in results/analysis/protocol_sensitivity through the accompanying supplementary package. The numerical evidence and accompanying scripts, including these calibration files, are archived at https://doi.org/10.5281/zenodo.22874266 (version v5). Raw histories are retained unchanged. Run experiments/analyze_protocol_sensitivity.py to regenerate the two new tables from the recorded CSV files. Recorded configurations and source hashes document the calibration. The reproduction instructions distinguish historical monitored runs from controlled count-only runs and preserve pilot failures. Tests check flux-energy identities, matched descriptors, reflection invariance and transpose equivalence. CPU-only tests explicitly skip unavailable CUDA tests; CUDA-required mode treats missing GPU support as an error. Timing reruns may differ with machine state.

References

[1] Virtanen P, Gommers R, Oliphant T E, Haberland M, Reddy T, Cournapeau D et al 2020 SciPy 1.0: fundamental algorithms for scientific computing in Python Nature Methods 17 261-272. doi:10.1038/s41592-019-0686-2
[2] Harris C R, Millman K J, van der Walt S J, Gommers R, Virtanen P, Cournapeau D et al 2020 Array programming with NumPy Nature 585 357-362. doi:10.1038/s41586-020-2649-2
[3] Lam S K, Pitrou A and Seibert S 2015 Numba: a LLVM-based Python JIT compiler Proc. Second Workshop on the LLVM Compiler Infrastructure in HPC 1-6. doi:10.1145/2833157.2833162
[4] Bell N, Olson L N, Schroder J and Southworth B 2023 PyAMG: algebraic multigrid solvers in Python Journal of Open Source Software 8 5495. doi:10.21105/joss.05495
[5] Strikwerda J C 2004 Finite Difference Schemes and Partial Differential Equations 2nd edn (Philadelphia: SIAM). doi:10.1137/1.9780898717938
[6] Hestenes M R and Stiefel E 1952 Methods of conjugate gradients for solving linear systems Journal of Research of the National Bureau of Standards 49 409-436. doi:10.6028/jres.049.044
[7] Saad Y 2003 Iterative Methods for Sparse Linear Systems 2nd edn (Philadelphia: SIAM). doi:10.1137/1.9780898718003
[8] Ruge J W and Stuben K 1987 Algebraic multigrid in McCormick S F (ed) Multigrid Methods (Philadelphia: SIAM) 73-130. doi:10.1137/1.9781611971057.ch4
[9] Bernaschi M, Celestini A, Richelli G and D'Ambra P 2026 On the energy efficiency of sparse matrix computations on multi-GPU clusters Future Generation Computer Systems 183 108519. doi:10.1016/j.future.2026.108519
[10] Welter A and Nguyen N C 2026 Preconditioning techniques for hybridizable discontinuous Galerkin discretizations on GPU architectures Computer Methods in Applied Mechanics and Engineering 456 118951. doi:10.1016/j.cma.2026.118951
[11] Yuan F, Yang X, Huang Y, Dong D, Xu C, Liu J et al 2025 CRAMG: a communication-reduced algebraic multigrid method Proc. 39th ACM International Conference on Supercomputing 397-411. doi:10.1145/3721145.3725764
[12] Green D, Hu X, Lore J, Mu L and Stowell M L 2022 An efficient high-order numerical solver for diffusion equations with strong anisotropy Computer Physics Communications 276 108333. doi:10.1016/j.cpc.2022.108333
[13] Koskela T, Christidi I, Giordano M, Dubrovska E, Quinn J, Maynard C et al 2023 Principles for automated and reproducible benchmarking Proc. SC '23 Workshops 609-618. doi:10.1145/3624062.3624133
[14] Standard Performance Evaluation Corporation 2021 734.hpgmgfv_m: SPEChpc 2021 Benchmark Description. https://www.spec.org/hpc2021/docs/benchmarks/734.hpgmgfv_m.html accessed 8 September 2026
[15] Lawrence Livermore National Laboratory n.d. AMG2023: ATS-6 Benchmarks documentation. https://software.llnl.gov/benchmarks/10_amg/amg.html accessed 8 September 2026
[16] D'Ambra P, Durastante F, Filippone S and Zikatanov L 2023 Automatic coarsening in algebraic multigrid utilizing quality measures for matching-based aggregations Computers & Mathematics with Applications 144 290-305. doi:10.1016/j.camwa.2023.06.026
[17] Anzt H, Cojean T, Flegar G, Göbel F, Grützmacher T, Nayak P et al 2022 Ginkgo: a modern linear operator algebra framework for high performance computing ACM Transactions on Mathematical Software 48 2:1-2:33. doi:10.1145/3480935
[18] Pekkila J, Lappi O, Robertsen F and Korpi-Lagg M J 2025 Stencil computations on AMD and Nvidia graphics processors: performance and tuning strategies Concurrency and Computation: Practice and Experience 37 e70129. doi:10.1002/cpe.70129
[19] Makhmut Y, Imankulov T, Gorlatch S and Matkerim B 2026 A CUDA performance study of global- and shared-memory kernels for the Buckley-Leverett polymer-flooding problem Applied Sciences 16 5449. doi:10.3390/app16115449
[20] Tsai Y-H M, Beams N and Anzt H 2023 Three-precision algebraic multigrid on GPUs Future Generation Computer Systems 149 280-293. doi:10.1016/j.future.2023.07.024
[21] Graillat S, Jézéquel F, Mary T and Molina R 2024 Adaptive precision sparse matrix-vector product and its application to Krylov solvers SIAM Journal on Scientific Computing 46 C30-C56. doi:10.1137/22M1522619

[22] Aksoylu B, Graham I G, Klie H and Scheichl R 2008 Towards a rigorously justified algebraic preconditioner for high-contrast diffusion problems Computing and Visualization in Science 11 319-331. doi:10.1007/s00791-008-0105-1
[23] Aksoylu B and Yeter Z 2010 Robust multigrid preconditioners for cell-centered finite volume discretization of the high-contrast diffusion equation Computing and Visualization in Science 13 229-245. doi:10.1007/s00791-010-0140-6
[24] Carson E, Liesen J and Strakos Z 2024 Towards understanding CG and GMRES through examples Linear Algebra and its Applications 692 241-291. doi:10.1016/j.laa.2024.04.003
[25] Firmbach M, Phillips M, Glusa C, Popp A, Siefert C M and Mayr M 2026 Smoothed aggregation algebraic multigrid for problems with heterogeneous and anisotropic materials arXiv:2602.05686 (preprint). doi:10.48550/arXiv.2602.05686
[26] Funk Y, Götz M and Anzt H 2022 Prediction of optimal solvers for sparse linear systems using deep learning Proc. 2022 SIAM Conference on Parallel Processing for Scientific Computing 14-24. doi:10.1137/1.9781611977141.2
[27] Sun H-B, Jing Y-F and Xu X-W 2025 A new matrix feature selection strategy in machine learning models for certain Krylov solver prediction Journal of Classification 42 72-89. doi:10.1007/s00357-024-09484-0
[28] Xiong H, Yang W, He W, Lin S, Li K and Li K 2025 MM-AutoSolver: a multimodal machine learning method for the auto-selection of iterative solvers and preconditioners Journal of Parallel and Distributed Computing 205 105144. doi:10.1016/j.jpdc.2025.105144
[29] Zabegaev Y, Keilegavlen E, Iversen E and Berre I 2024 Automated linear solver selection for simulation of multiphysics processes in porous media Computer Methods in Applied Mechanics and Engineering 426 117031. doi:10.1016/j.cma.2024.117031
[30] Patankar S V 1980 Numerical Heat Transfer and Fluid Flow 1st edn (New York: Hemisphere). ISBN:9780891165224
[31] Hoefler T and Belli R 2015 Scientific benchmarking of parallel computing systems: twelve ways to tell the masses when reporting performance results Proc. International Conference for High Performance Computing, Networking, Storage and Analysis (SC15) 73:1-73:12. doi:10.1145/2807591.2807644